《AI 秘书 – 基于大模型的智能个人助理》
部署文档（Deployment Specification）

软件工程课程项目组

2026 年 4 月 20 日

目录

1 引言 **3**
1.1 文档目的 3
1.2 适用范围 3
1.3 代码基线与说明 3
1.4 代码现状与需求文档差异说明 4

2 系统部署对象概述 **4**
2.1 系统形态 4
2.2 模块职责划分 4
2.3 页面路由关系 4

3 部署环境设计 **6**
3.1 运行环境要求 6
3.2 代码侧依赖能力 6
3.3 权限与配置建议 7
3.4 外部服务依赖说明 7

4 数据存储与初始化机制 **8**
4.1 数据库文件与安全级别 8
4.2 数据表结构 8
4.3 数据库迁移策略 8
4.4 启动初始化时序 9

5 部署前准备 **9**
5.1 工程文件检查清单 9
5.2 模型配置准备 9
5.3 账号与数据清理准备 10

6 标准部署流程 **10**
6.1 方式一：通过 DevEco Studio 图形界面部署 10
6.2 方式二：命令行构建（在工程已具备 wrapper 时） 10
6.3 真机部署特别说明 11
6.4 首轮人工验收步骤 11

7 部署后的运行验收 **11**
7.1 普通会话验收点 11
7.2 心理辅导会话验收点 12
7.3 本地持久化验收点 12

8 安全与发布建议 **12**
8.1 当前版本安全风险 12
8.2 发布前加固建议 13
8.3 发布检查表 13

9 常见故障与排查 **14**
9.1 应用启动后白屏或无法进入页面 14
9.2 模型调用一直超时或无响应 14
9.3 会话存在但消息/日程看不到 15
9.4 心理辅导页没有切换成功 15

10 运维与后续扩展建议 **15**
10.1 建议保留的最小运维动作 15
10.2 后续扩展方向 16

11 附录：部署执行清单 **16**

1 引言

1.1 文档目的

本文档面向课程项目组成员、助教/评审、后续维护者，给出《AI 秘书 – 基于大模型的智能个人助理》项目的**部署前提**、**环境要求**、**配置步骤**、**初始化流程**、**发布检查项与故障排查方案**。本文档的目标不是抽象描述理想架构，而是基于当前已上传的 ArkTS 源码，对项目的**真实可运行形态**进行可复现实操说明，确保接手者能够在标准 HarmonyOS 工程中完成导入、编译、安装、配置、首轮验收与后续维护。

1.2 适用范围

本文档覆盖以下范围：

- HarmonyOS Stage 模型应用的本地部署；
- ArkTS 页面、数据库、执行体、安全检查、LLM 调用链路的配置与联调；
- 模拟器与真机两类运行环境；
- 本地 relationalStore 数据初始化、会话恢复、首轮业务验收；
- 开发演示环境与课程答辩环境下的最小可行部署。

本文档**不覆盖**以下内容：应用市场正式上架流程、CI/CD 平台搭建、企业级灰度发布、服务端网关建设、自动化设备农场、云端日志平台等超出课程项目边界的内容。

1.3 代码基线与说明

本部署文档依据当前上传源码进行编写，核心基线包括：

- 页面与组件：`Login.ets`、`SessionSelect.ets`、`Index.ets`、`PsychChat.ets`、`ChatDetail.ets`、`ActionDialog.ets`；
- 业务与持久化：`DatabaseManager.ets`、`ReadExecutor.ets`、`WriteExecutor.ets`、`SecurityChecker.ets`、`LLMService.ets`；
- 启动入口：`EntryAbility.ets`；
- 格式参考：已上传需求文档 `requirement.tex` / `requirement.pdf`。

重要说明：当前会话中未上传完整工程级文件（如 `module.json5`、`build-profile.json5`、`resources/base/profile` 下的页面路由声明、签名配置、hvigor 构建脚本等），因此本文档在工程级部署步骤部分采用**标准 HarmonyOS Stage 工程假设**进行编写，并对需要由项目组自行补齐的内容单独标注。

1.4 代码现状与需求文档差异说明

为保证文档可信度，部署必须以**代码现状**为准。当前实现与需求文档存在以下值得注意的差异：

1. **模型服务差异**：需求文档描述为上海交通大学 GLM-5 / `models.sjtu.edu.cn`，而当前 `LLMService` 默认实现使用的是 `https://open.bigmodel.cn/api/paas/v4/chat/completions`，模型名为 `glm-4-flash`，并提供 `setAPIConfig()` 作为替换入口。
2. **离线降级触发机制差异**：当前代码具备异常后的本地 Mock 降级，但尚未看到显式网络类型探测逻辑；因此“断网 $<$ 500ms 自动切换”在当前版本中**不宜直接宣称已完全满足**，需结合实际网络层报错速度验证。
3. **密钥与凭据安全性**：当前 `LLMService` 中存在硬编码 API Key；当前 `Users` 表以明文形式保存密码。此做法适合课程演示，不适合生产环境。
4. **敏感操作确认范围**：模型驱动的 `schedule` 操作已经落地为确认弹窗；`delete/modify/share/purchase` 目前主要仍作为预留动作名单存在，尚未形成完整的统一确认流。

2 系统部署对象概述

2.1 系统形态

本项目是一个运行在 HarmonyOS 端侧的 ArkTS 声明式 UI 应用，采用“**本地数据库 + 读写执行体 + 模型路由 + 安全检查 + 多页面会话界面**”的结构化实现。项目运行后，用户将经历以下基本链路：

1. **启动判断**：`EntryAbility` 初始化数据库并读取登录态；
2. **账号进入**：未登录进入 `Login`，已登录进入 `SessionSelect`；
3. **会话选择**：用户选择普通会话或心理辅导会话；
4. **对话执行**：普通会话进入 `Index`，心理辅导进入 `PsychChat`；
5. **业务流转**：输入经过 `ReadExecutor` 组装 Prompt，经 `LLMService` 调用模型，再由 `SecurityChecker` 与 `WriteExecutor` 决定落地动作；
6. **持久化**：消息、日程、会话及运行时状态由 `DatabaseManager` 写入本地 `relationalStore`。

2.2 模块职责划分

2.3 页面路由关系

结合源码逻辑，当前项目的主要页面路由关系如下：

模块	类型	职责说明
EntryAbility.ets	启动入口	初始化数据库，判断是否已登录，决定首屏加载 Login 或 SessionSelect
Login.ets	页面	处理注册/登录、输入校验、登录态恢复
SessionSelect.ets	页面	会话列表展示、新建会话、删除会话、切换会话、退出登录
Index.ets	页面	普通会话主页面，承载聊天、学习建议、日程汇总、Tab 导航与日程编辑
PsychChat.ets	页面	独立心理辅导页，采用单独主题色与独立消息流
ChatDetail.ets	页面	轻量级历史聊天详情页，主要做记录浏览
ActionDialog.ets	组件	对敏感操作（日程添加）进行二次确认
DatabaseManager.ets	数据层	建表、迁移、登录态持久化、消息/日程/会话 CRUD、会话标题生成
ReadExecutor.ets	读执行体	读取最近 5 条消息、设备信息、电量信息并拼装 Prompt
WriteExecutor.ets	写执行体	解析模型 JSON、生成日程结构、学习建议、心理状态评估
SecurityChecker.ets	安全层	检查模型输出 JSON 结构、安全关键词、隐私泄露、提示词注入、敏感动作
LLMService.ets	网络/模型层	调用云端大模型或降级到本地 Mock 返回

表 1: 部署对象模块说明表

$$\texttt{EntryAbility} \rightarrow \begin{cases} \texttt{pages/Login}, & \text{未登录} \\ \texttt{pages/SessionSelect}, & \text{已登录} \end{cases} \tag{1}$$

之后的分流关系为：

- `SessionSelect` → `pages/Index`（普通会话）
- `SessionSelect` → `pages/PsychChat`（心理辅导会话）
- `Index` 内部检测到当前会话类型为 `psych` 时，会自动重定向到 `pages/PsychChat`
- `PsychChat` 返回会话中心时跳转 `pages/SessionSelect`

因此,部署时必须保证页面注册、资源路径与源码中的 `router.pushUrl()` / `router.replaceUrl()` 路径保持一致。

3 部署环境设计

3.1 运行环境要求

根据当前源码能力，推荐的运行环境如下：

类别	要求	说明
开发 IDE	DevEco Studio	用于导入 HarmonyOS ArkTS 工程、编译与真机/模拟器运行
应用模型	Stage 模型	`EntryAbility` 使用 Stage 模型生命周期接口
语言/前端框架	ArkTS + ArkUI	源码全部为 `.ets` 页面与组件
本地数据库	relationalStore	存储消息、会话、日程、用户、运行时设置
网络能力	HTTP 出网	云端模型模式下必须可访问 LLM API 地址
终端环境	模拟器或真机	推荐至少准备 1 台真机用于网络/性能/断网验证

表 2: 基础运行环境要求

3.2 代码侧依赖能力

从 import 语句看，当前代码依赖以下 HarmonyOS 能力模块：

- `@kit.AbilityKit`：Ability 生命周期管理；
- `@kit.ArkUI`：UI 组件、路由、窗口管理；
- `@ohos.data.relationalStore`：RDB 数据存储；
- `@ohos.net.http`：HTTP 请求；
- `@ohos.batteryInfo`：读取电量百分比；
- `@ohos.deviceInfo`：读取设备型号；
- `@ohos.app.ability.common`：页面上下文与数据库上下文注入。

因此，部署时必须确保工程所引用的 SDK 已经包含以上模块，并且 API 级别与项目模板兼容。

3.3 权限与配置建议

根据当前代码实现，**最低建议权限**如下：

1. **必须**：`ohos.permission.INTERNET`，用于调用远端 LLM 接口；
2. **建议预留**：`ohos.permission.GET_NETWORK_INFO`，便于后续实现显式网络状态探测与更快的离线降级；
3. **禁止滥申**：不应申请与当前功能无关的高敏权限，如位置、相机、通讯录、麦克风等。

若项目组需要在答辩时展示“断网快速降级”的可靠性，建议补充网络状态判断并在 `LLMService` 中显式切换 `useMock`。

3.4 外部服务依赖说明

当前版本的模型服务默认配置如下：

```
private apiUrl: string = 'https://open.bigmodel.cn/api/paas/v4/chat/completions'
private apiKey: string = '***当前源码中为硬编码明文***'
private model: string = 'glm-4-flash'
private useMock: boolean = false
```

Listing 1: LLMService 默认模型配置（代码现状）

部署含义如下：

- 若网络可用且 API Key 正常，应用优先走云端模型；
- 若 HTTP 非 200、响应解析失败或请求抛出异常，将在 `catch` 中降级为本地 Mock；
- 若课堂环境网络不稳定，可在初始化阶段主动调用 `setMockMode(true)` 进入纯离线演示模式；
- 若要与需求文档中的其他模型网关对齐，可使用 `setAPIConfig(url, key, model)` 动态替换。

部署建议：演示环境可接受源码内临时写死 Key；正式交付环境必须将 API Key 移出源码，至少迁移到本地配置文件、构建时环境变量或受控服务端网关。

4 数据存储与初始化机制

4.1 数据库文件与安全级别

`DatabaseManager` 中的数据库配置如下：

```
const STORE_CONFIG: relationalStore.StoreConfig = {
  name: 'AIAssistant.db',
  securityLevel: relationalStore.SecurityLevel.S1
}
```

Listing 2: 数据库配置

即应用在本地创建名为 `AIAssistant.db` 的关系型数据库文件，并使用 S1 级别安全策略。

4.2 数据表结构

当前代码在首次启动时会自动创建以下数据表：

表名	主键	主要字段
Messages	id	conversationId, content, isUser, timestamp
Schedules	id	conversationId, content, createdTime, remindTime, isDone
Users	id	username, password, createdTime
AppSettings	key	value（如 currentUserId、currentUsername、currentConversationId）
Conversations	id	userId, title, type, parentConversationId, createdTime, updatedTime, lastMessage

表 3: 部署后自动初始化的数据表

4.3 数据库迁移策略

代码采用“先建表、后补列”的向前兼容策略：

1. 若数据库不存在，则创建全量业务表；
2. 若数据库已存在，则通过 `safeAddColumn()` 对旧表尝试补齐缺失列；
3. 若补列时报“列已存在”类异常，则忽略，从而避免旧版本数据库无法启动。

这意味着部署过程中即使用户已存在旧库，应用通常也可以平滑升级到较新的字段结构。

4.4 启动初始化时序

建议将启动时序理解为以下四段：

1. **Ability 级初始化**：`EntryAbility.onWindowStageCreate()` 中注入上下文并初始化数据库；
2. **登录态恢复**：读取 `AppSettings` 中的 `currentUserId/currentUsername`；
3. **会话恢复**：读取 `currentConversationId` 决定进入普通会话或心理会话；
4. **页面数据装载**：对应页面再加载最近消息与当前会话日程。

因此，部署完成后的**首轮验收重点**之一就是检查：首次安装、二次启动、登录后退出再进入、切换会话再重启，是否都能正确恢复状态。

5 部署前准备

5.1 工程文件检查清单

在正式编译前，请先确认以下工程级文件与声明已经补齐：

1. **页面注册**：`pages/Login`、`pages/SessionSelect`、`pages/Index`、`pages/PsychChat`、`pages/ChatDetail` 已在页面配置中正确声明；
2. **入口 Ability**：`EntryAbility.ets` 已在应用配置中声明为入口 Ability；
3. **权限声明**：至少声明 `INTERNET`；若要做显式离线检测则补充 `GET_NETWORK_INFO`；
4. **签名与产品配置**：已配置本地调试签名；
5. **资源目录**：若页面引用自定义资源（图标、字符串、profile），需确保资源不缺失；
6. **构建脚本**：若使用命令行构建，确保 hvigor wrapper 存在并可执行。

5.2 模型配置准备

建议部署前明确两套运行模式：

- **联机演示模式**：保留真实 API 地址，并验证 Key 可用；
- **离线答辩模式**：在应用初始化或页面构造期间打开 `setMockMode(true)`，避免课堂网络波动导致展示失败。

推荐将演示时的模型配置抽离为单独方法，例如：

```
private initLLM(): void {
  // 答辩演示可强制使用本地 mock
  this.llmService.setMockMode(true)

  // 若需要恢复真实模型，可使用：
  // this.llmService.setAPIConfig(url, key, model)
}
```

Listing 3: 建议的模型配置初始化方式

5.3 账号与数据清理准备

为了保证部署后的验收结果干净、可复现，建议在安装前明确测试数据策略：

1. 首次验收前使用**全新安装**或清除应用数据；
2. 若要验证“数据库迁移与会话恢复”，则保留旧库进行增量启动；
3. 若同一设备多人轮流答辩，建议在每轮演示后执行**退出登录 + 清理应用数据**。

6 标准部署流程

6.1 方式一：通过 DevEco Studio 图形界面部署

推荐优先采用图形界面部署，步骤如下：

1. 打开 DevEco Studio，选择“Open”导入项目根目录；
2. 等待 IDE 完成索引、SDK 检查与依赖同步；
3. 打开工程配置，确认入口 Ability、页面路由、签名配置、权限声明完整；
4. 检查 `LLMService.ets`，确认当前使用的运行模式（真实 API / Mock）；
5. 选择目标设备（模拟器或真机）；
6. 执行 Build/Run；
7. 等待 HAP 安装完成并自动启动应用；
8. 观察首屏是否正确进入 Login 或 SessionSelect。

6.2 方式二：命令行构建（在工程已具备 wrapper 时）

若项目根目录包含 hvigor wrapper，则可补充命令行构建流程；但由于当前会话未上传完整工程级构建文件，本文仅给出**部署规范而非硬编码命令**：

1. 确认本地 Java、Node、SDK、签名配置已完整；

2. 执行工程标准的 assemble/build 命令生成 HAP；
3. 使用标准安装工具将 HAP 部署到设备；
4. 首次运行后观察数据库初始化与登录页跳转是否成功。

6.3 真机部署特别说明

真机部署时建议额外完成以下检查：

- 真实网络请求是否可访问模型服务；
- 切换飞行模式后应用是否会阻塞过久；
- 页面切换、滚动与输入框行为是否比模拟器更稳定；
- 电量和设备型号字段是否能被 `ReadExecutor` 正常读取。

6.4 首轮人工验收步骤

应用成功启动后，建议按照以下顺序做“最小闭环验收”：

1. 注册新用户并自动登录；
2. 进入会话中心，新建普通会话；
3. 在普通会话输入“帮我安排明天下午 3 点在会议室开会”；
4. 检查是否弹出 `ActionDialog`，确认后日程是否写入列表；
5. 输入“我想学 Python”，检查是否返回学习建议；
6. 输入“最近压力有点大”，检查是否切换/创建心理辅导会话；
7. 退出应用后重新打开，检查登录态与当前会话是否恢复。

若以上 7 步全部通过，则可以判定本地部署基本成功。

7 部署后的运行验收

7.1 普通会话验收点

普通会话部署成功后应具备以下表现：

1. 顶部显示当前会话标题，且首条用户消息后标题自动变得更语义化；
2. 底部具备“对话 / 通知 / 我的”三栏切换；

3. 聊天气泡左右分布正确：AI 在左、用户在右；
4. 普通消息可直接写入会话；
5. 学习类消息会生成固定结构化回复；
6. 日程类消息会先弹确认框再入库；
7. 日程支持编辑、删除、标记完成；
8. “我的”页可切换会话、退出登录。

7.2 心理辅导会话验收点

心理辅导部署成功后应具备以下表现：

1. 心理会话与普通会话数据隔离；
2. 进入心理页时顶部与背景为紫色系主题；
3. Prompt 会自动补入“请用心理支持和温和陪伴的方式回应”；
4. 情绪关键词可触发 `WriteExecutor` 的分级建议；
5. 若模型输出被安全检查拦截，页面会退化为温和的本地兜底回复。

7.3 本地持久化验收点

部署完成后建议校验以下本地持久化能力：

- 注册用户后，`Users` 表中新增记录；
- 登录成功后，`AppSettings` 中写入 `currentUserId/currentUsername`；
- 创建会话后，`Conversations` 表写入会话；
- 发送消息后，`Messages` 表按当前会话隔离写入；
- 创建日程后，`Schedules` 表按会话隔离写入；
- 删除普通会话时，其附属心理会话、消息与日程被级联删除。

8 安全与发布建议

8.1 当前版本安全风险

基于上传源码，当前版本存在以下部署风险：

1. **API Key 硬编码**：源码泄露后可直接被复用；

2. **密码明文存储**：`Users.password` 直接保存明文；
3. **离线切换不够主动**：未见显式网络状态探测，弱网下可能等待较长；
4. **用户输入侧前置安全拦截较弱**：当前 `SecurityChecker` 主要审查模型输出；
5. **删除/修改操作确认流不统一**：本地 UI 中的删除、编辑是直接执行的。

8.2 发布前加固建议

若项目需要从“课程演示版”提升到“可交付演示版”，建议至少完成以下加固：

1. 将 API Key 改为环境变量或安全配置注入；
2. 将用户密码改为哈希后存储，禁止明文比对；
3. 增加显式网络状态探测与超时快速熔断；
4. 为删除日程、删除会话等操作增加确认弹窗；
5. 对用户输入在发送到 LLM 之前也做本地敏感词与越权检查；
6. 对异常日志进行统一脱敏，避免把敏感 Prompt 打到控制台。

8.3 发布检查表

检查项	是否必须	验收标准
入口 Ability 正常加载	Must	首屏能稳定进入 Login 或 SessionSelect
数据库初始化成功	Must	首次启动不报错，能创建本地库与 5 张业务表
页面路由完整	Must	Login / SessionSelect / Index / PsychChat 路由均可达
网络权限声明完整	Must	联机模式下模型请求可以正常发出
会话切换正确	Must	普通会话与心理会话进入路径正确
日程确认弹窗生效	Must	schedule 动作进入 ActionDialog，取消时不落库
Mock 降级可用	Must	API 异常时仍能返回可解析 JSON 并继续业务流
登录态恢复正常	Must	重启后可恢复到最近登录用户与当前会话
API Key 外置化	Should	不在版本库中保存明文 Key
密码哈希存储	Should	Users 表中不保存明文密码

检查项	是否必须	验收标准
删除/修改确认统一化	Should	会话/日程删除前均有显式确认
日志脱敏	Should	控制台不输出敏感用户数据与密钥

9 常见故障与排查

9.1 应用启动后白屏或无法进入页面

可能原因：

- 入口 Ability 未配置；
- 页面路径注册与源码中的 `pages/xxx` 不一致；
- 数据库初始化失败后未正确处理页面状态；
- 工程资源配置缺失导致构建虽然成功但运行失败。

排查建议：

1. 检查入口 Ability 与页面注册配置；
2. 查看控制台是否出现 `DatabaseManager context is not set` 或 `Database is not ready`；
3. 确认页面文件路径、类名与路由 URL 完全一致。

9.2 模型调用一直超时或无响应

可能原因：

- API 地址不可达；
- API Key 无效或额度受限；
- 当前网络环境限制出网；
- 设备断网但未显式启用 Mock，HTTP 层等待较久。

排查建议：

1. 先将 `LLMService` 切换到 `setMockMode(true)`，验证本地链路是否正常；
2. 检查 `responseCode` 和异常日志；
3. 检查权限声明是否包含 `INTERNET`；
4. 若答辩环境网络不稳定，建议直接使用 Mock 模式。

9.3 会话存在但消息/日程看不到

可能原因：

- 当前 `conversationId` 未被正确设置；
- 启动后恢复到了错误的当前会话；
- 数据已写入旧会话，而页面读取的是新会话；
- 删除会话后 `currentConversationId` 未及时刷新。

排查建议：

1. 检查 `AppSettings.currentConversationId`；
2. 检查 `Messages/Schedules.conversationId` 是否与当前会话一致；
3. 在 `SessionSelect` 中重新进入目标会话。

9.4 心理辅导页没有切换成功

可能原因：

- 当前动作未被模型识别为 `psych`；
- 路由未注册 `pages/PsychChat`；
- `getOrCreatePsychConversation()` 依赖当前用户或会话状态时发生失败。

排查建议：

1. 在普通会话先确认用户已登录且当前会话存在；
2. 使用包含“焦虑/压力/难过”等明确情绪词的输入进行验证；
3. 检查数据库中是否已创建 type=`psych` 的会话。

10 运维与后续扩展建议

10.1 建议保留的最小运维动作

即使是课程项目，也建议保留以下最小运维动作：

1. 每次演示前清理无效测试数据；
2. 记录当前模型配置（真实 API / Mock）；
3. 固定一套标准验收脚本，避免每次人工操作不同；
4. 对数据库升级策略保持向前兼容，避免旧机上的历史数据不可读。

10.2 后续扩展方向

部署结构上，当前项目已经具备较好的扩展位：

- 在 `WriteExecutor` 中继续新增 `weather/translation/search` 等 Action；
- 在 `SecurityChecker` 中细化用户输入与模型输出双向检查；
- 在 `DatabaseManager` 中新增心理日志表或用户偏好表；
- 在 `LLMService` 中接入多模型策略、重试、缓存与更细粒度熔断。

11 附录：部署执行清单

序号	执行项	完成标准
1	导入工程	DevEco Studio 能正常索引工程，无关键红线报错
2	检查页面配置	Login / SessionSelect / Index / PsychChat 已注册
3	检查入口 Ability	EntryAbility 为应用启动入口
4	检查权限	至少声明 INTERNET；按需声明 GET_NETWORK_INFO
5	检查模型配置	明确当前使用真实 API 还是 Mock
6	构建并安装	应用可成功安装到模拟器或真机
7	首次启动验收	能进入 Login 页面并完成注册/登录
8	会话验收	能创建普通会话与心理辅导会话
9	日程验收	输入日程语句后可弹窗并成功落库
10	学习验收	输入学习类语句后可生成本地结构化建议
11	心理验收	输入情绪类语句后可切换心理会话并返回温和回复
12	重启验收	重启后可恢复登录态与当前会话
13	安全检查	异常模型输出不会导致应用崩溃
14	发布前加固	至少处理 API Key 和明文密码问题